

[--- CODE CELL ---]

```
from sklearn.datasets import load_breast_cancer
import pandas as pd
```

[--- CODE CELL ---]

```
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
```

[--- CODE CELL ---]

```
import matplotlib.pyplot as plt
```

[--- CODE CELL ---]

```
import seaborn as sns
```

[--- CODE CELL ---]

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

#we need accuracy score to predict if the values are correct or not
#the same formula that we have for True_positive and True_negative
```

[--- CODE CELL ---]

```
X = df.drop('target', axis = 1)
y = df['target']
```

[--- CODE CELL ---]

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
```

[--- CODE CELL ---]

```
knn = KNeighborsClassifier(n_neighbors = 5)
knn.fit(X_train, y_train)
```

[--- CODE CELL ---]

```
y_pred = knn.predict(X_test)
accuracy_score(y_test, y_pred)
```

[--- CODE CELL ---]

```
#all the accuracies caluculated
# accuracy 1: calculated was: 0.956140350877193
```

[--- CODE CELL ---]

```
# Now we scale the data
```

[--- CODE CELL ---]

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
```

[--- CODE CELL ---]

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[--- CODE CELL ---]

```
knn = KNeighborsClassifier(n_neighbors = 5)
knn.fit(X_train_scaled, y_train)
```

[--- CODE CELL ---]

```
y_pred = knn.predict(X_test_scaled)
accuracy_score(y_test, y_pred)
```

[--- CODE CELL ---]

```
#all the accuracies caluculated after scaling
# accuracy 1: calculated was: 0.9473684210526315
```

[--- CODE CELL ---]

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

[--- CODE CELL ---]

```
def knn_k_accuracy(X_train, X_test, y_train, y_test, max_k = 20):
    """
    Tries different values of K for KNN and returns accuracy for each K.
    Parameters:
    -----
    X_train, X_test : scaled feature matrices
    y_train, y_test : target values
    max_k : maximum K to test
    Returns:
    -----
    k_values : list of K values
    accuracies : list of accuracy scores
    """
    k_values = []
    accuracies = []
    for k in range(1, max_k + 1):
        knn = KNeighborsClassifier(n_neighbors = k)
        knn.fit(X_train, y_train)
        y_pred = knn.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
        k_values.append(k)
        accuracies.append(acc)
    return k_values, accuracies
```

[--- CODE CELL ---]

```
k_vals, accs = knn_k_accuracy(
    X_train_scaled,
    X_test_scaled,
    y_train,
    y_test,
    max_k = 20
)
for k, acc in zip(k_vals, accs):
    print(f"k = {k:2d} | Accuracy = {acc:.4f}")
```

[--- CODE CELL ---]

[--- CODE CELL ---]

```
#visualisi ng accuracy with K
```

[--- CODE CELL ---]

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
plt.plot(k_vals, accs, marker = 'o')
plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy vs K")
plt.show()
```

[--- CODE CELL ---]

```
best_k = k_vals[accs.index(max(accs))]
print(f"Best K = {best_k}")
```

[--- CODE CELL ---]

```
#just making sure the predicted value is same everywhere
```

[--- CODE CELL ---]

```
knn = KNeighborsClassifier(n_neighbors=9)
knn.fit(X_train_scaled, y_train)
```

[--- CODE CELL ---]

```
y_pred_9 = knn.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred_9)
```

[--- CODE CELL ---]

```
acc
```

[--- CODE CELL ---]